Report
Version Control System from Scratch

Name: R P Pranav

SRN: PES2UG24CS917

Sec : C

Email: rppranav081@gmail.com

Objective: Build a local version control system that tracks file changes, stores snapshots efficiently, and supports commit history. Every component maps directly to operating system and filesystem concepts.

Platform: Ubuntu 24.04 (Parallels/Ubuntu-gnu-linux)

Github Repo Link: https://github.com/pranav-codes55/PES2UG24CS917-pes-vcs

Phase 1:

```
parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$
```

```
parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$ ./pes init
find .pes/objects -type f
Initialized empty PES repository in .pes/
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$
```

Phase 2:

```
parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$ make test_tree
./test_tree
make: 'test_tree' is up to date.
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

```
parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5
e5c4b9aafc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$
```

Phase 3:

3a)

```
Apr 20 21:56
parallels@ubuntu-gnu-linux-24-04-3: ~/Desktop/PES2UG24CS917-pes-vcs

parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$ make clean
make pes

./pes init
echo hello > file1.txt
./pes add file1.txt
./pes status
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_read':
object.c:207:5: warning: ignoring return value of 'fread' declared with attribute 'warn_unused_result' [-Wunused-result]
  207 |     fread(buf, 1, size, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.h
  untracked:  commit.h
  untracked:  README.md
  untracked:  .gitignore
  untracked:  pes.h
  untracked:  index.h
  untracked:  test_tree.c
  untracked:  commit.c
  untracked:  test_objects.c
  untracked:  Makefile
```

```
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_read':
object.c:207:5: warning: ignoring return value of 'fread' declared with attribute 'warn_unused_result' [-Wunused-result]
  207 |     fread(buf, 1, size, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.h
  untracked:  commit.h
  untracked:  README.md
  untracked:  .gitignore
  untracked:  pes.h
  untracked:  index.h
  untracked:  test_tree.c
  untracked:  commit.c
  untracked:  test_objects.c
  untracked:  Makefile
  untracked:  pes.c
  untracked:  file2.txt
  untracked:  test_sequence.sh
  untracked:  index.c
  untracked:  object.c
  untracked:  tree.c

parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$
```

3b)

```
parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776702385 6 file1.txt
parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$
```

4

a)

```
parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$ ./pes log
commit 0e11134587c060f06a7d0de72c6af1d7192c833551d454f204ad27c93949bd65
Author: R P Pranav <rppranav081@gmail.com>
Date:   1776707239

    Third commit!

commit eb5bf6c5417925b03dfb1ff6b83039d605bfd40b15b194802e25dd5cfa5b5469
Author: R P Pranav <rppranav081@gmail.com>
Date:   1776707239

    Second commit

commit d286a55a7ce2fc7579109bdefd57da31cb651893803038b47f6f3407e287844e
Author: R P Pranav <rppranav081@gmail.com>
Date:   1776707201

    Initial commit with correct authorre1

parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$
```

b)

```
parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/0e/7ec74d27a9667fc3b02df7b2e22d58766bf925b534593b1f6847ad9fce30c4
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/39/64a974d3c49f25f64675cf663509925a760d9f31e7e6c1619c1a3dbd5ca45f
.pes/objects/65/3c38c51dcbbf90b2cfde63a1a447e738700964d0bb28c4fe1627eddb613800
.pes/objects/72/0deb36cb1b58fe9b9186b87422d29f5e0aae168e55b9a7cf58beb24177fecb
.pes/objects/72/5fcc4e398378423b92e9d31fb67808b3a99c59126336c89a9d6d792f082cff
.pes/objects/74/4d52b294402826ee144dbae8905ec338fea014b2f3e80a86f2486ea48e8488
.pes/objects/77/53500f7ce8bae4e9b8450c0da7a372f2fda7c4d0499191603a777748a0fca3
.pes/objects/f2/bff1edf58f9ab3c2da22921747d504b737f5acc1cd780f541b0e06775505de
.pes/objects/fc/4cf793b9282a0e61ef35215ee1332919f5c2fc8501b07322f0f3f9c9e94ebb
.pes/refs/heads/main
parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$
```

c)

```
parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
fc4cf793b9282a0e61ef35215ee1332919f5c2fc8501b07322f0f3f9c9e94ebb
ref: refs/heads/main
parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$
```

Phase 5 & 6: Analysis-Only Questions

The following questions cover filesystem concepts beyond the implementation scope of this lab. Answer them in writing — no code required.

Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what

files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

Ans:

A branch is stored as a file in:

.pes/refs/heads/<branch>

Steps to implement checkout:

1. Check if the branch exists in .pes/refs/heads/
2. Read the commit hash from that branch file
3. Load the commit object and get its tree
4. Traverse the tree:
    - Read each blob using object_read
    - Recreate files in the working directory
5. Remove files that exist in the current working directory but not in the new tree

6. Update .pes/HEAD to:
   ref: refs/heads/<branch>

What makes it complex:
• Handling nested directories recursively
• Avoiding overwriting uncommitted changes
• Managing untracked files safely

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

A dirty working directory means files have been modified but not committed.

Steps:

1. Load the index using index_load
2. For each tracked file:
    - Use stat() to get current file metadata (size, mtime)
    - Compare with index values
3. If metadata differs:
    - Recompute hash of file
    - Compare with stored hash in index

4. If hashes differ → file is modified

Decision:

- If file is modified AND differs in target branch → reject checkout

Key idea:

A file is dirty if:

working directory file ≠ index entry

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Normally:

HEAD → refs/heads/main

In detached HEAD:

HEAD → <commit hash>

- New commits are created
- But they are not attached to any branch
- So they can become unreachable

1.Find commit using:

pes log

2. Create a new branch manually:

- Create file:

.pes/refs/heads/recovery

- Write commit hash into it

  Key idea:

  Detached HEAD commits exist but are not referenced by any branch

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Remove objects not reachable from any branch

Algorithm:

1. Start from all branch references:

.pes/refs/heads/*

2. For each branch:
   - Read commit hash
   - Traverse commit history (parent chain)
3. For each commit:
   - Add commit hash to set
   - Add its tree hash
4. Traverse trees recursively:
   - Add all subtree hashes
   - Add all blob hashes
5. Store all reachable hashes in a hash set
6. Scan `.pes/objects/`:
   - If object hash NOT in set → delete it

Data structure:

Use a hash set

- Fast lookup (O(1))
- Avoid duplicates

Estimate:

For:

- 100,000 commits
- ~10 objects per commit

Total objects ≈ 1,000,000

This ensures only reachable objects are preserved while unused objects are safely removed.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Problem: Race condition

Scenario:

1. Commit process writes a new object
2. Object is not yet referenced
3. GC runs at the same time
4. GC sees object as unreachable
5. GC deletes the object
6. Commit later references deleted object

Result:

Repository becomes corrupted

How Git avoids this:

- Uses grace period
    - Recent objects are not deleted
- Uses file timestamps (mtime)
- Uses locking mechanisms
- GC runs only on safe objects

Key idea:

Never delete objects that might still be used by an ongoing commit

Thus, synchronization and safety mechanisms are required to prevent data loss.

Final Integration Test

```
parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: d83deff758ab... Initial commit

Log after first commit:
commit d83deff758abe4b6eabc220441fb247128a69bb6736b2543f5d5bd32cd51b698
Author: R P Pranav <rppranav081@gmail.com>
Date:   1776706629

    Initial commit
```

```
Date:   1776706629

    Initial commit


--- Second Commit ---
Committed: 70194abc7bb1... Update file.txt

--- Third Commit ---
Committed: 7ef42ca46a88... Add farewell

--- Full History ---
commit 7ef42ca46a881bd047f1350213d45156260434f0ee02ec49fe7efd8ac1f8d888
Author: R P Pranav <rppranav081@gmail.com>
Date:   1776706629

    Add farewell!

commit 70194abc7bb15da8647f62ea48f04ca80370ae56e6ed34455ca0d0d420dec0e4
Author: R P Pranav <rppranav081@gmail.com>
Date:   1776706629

    Update file.txt

commit d83deff758abe4b6eabc220441fb247128a69bb6736b2543f5d5bd32cd51b698
Author: R P Pranav <rppranav081@gmail.com>
Date:   1776706629

    Initial committial cA


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
7ef42ca46a881bd047f1350213d45156260434f0ee02ec49fe7efd8ac1f8d888
```

```
    Add farewell!

commit 70194abc7bb15da8647f62ea48f04ca80370ae56e6ed34455ca0d0d420dec0e4
Author: R P Pranav <rppranav081@gmail.com>
Date:   1776706629

    Update file.txt

commit d83deff758abe4b6eabc220441fb247128a69bb6736b2543f5d5bd32cd51b698
Author: R P Pranav <rppranav081@gmail.com>
Date:   1776706629

    Initial committial cA


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
7ef42ca46a881bd047f1350213d45156260434f0ee02ec49fe7efd8ac1f8d888

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/70/194abc7bb15da8647f62ea48f04ca80370ae56e6ed34455ca0d0d420dec0e4
.pes/objects/7e/f42ca46a881bd047f1350213d45156260434f0ee02ec49fe7efd8ac1f8d888
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/d8/3deff758abe4b6eabc220441fb247128a69bb6736b2543f5d5bd32cd51b698
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
parallels@ubuntu-gnu-linux-24-04-3:~/Desktop/PES2UG24CS917-pes-vcs$
```